Guia Complementar: Componentes da Aula 6 (Não usados nos projetos)

Como declarar, configurar, ler valores e conectar eventos seguindo o padrão de código da disciplina

---

1. Texto e Formatação (`MaskedTextBox`, `RichTextBox`, `LinkLabel`)

1.1. `MaskedTextBox` (Entrada com formato/máscara fixa)

- **Finalidade (Slide 3):** Receber entradas com formato rigoroso, como CEP, CPF, telefone ou data, impedindo que o usuário digite letras onde só cabem números.
- **Caracteres de Máscara no Windows Forms:**
  - `0` = Dígito numérico obrigatório (`0` a `9`).
  - `9` = Dígito numérico ou espaço opcional.
  - `L` = Letra obrigatória (`a-z`, `A-Z`).
  - `>` = Converte todas as letras seguintes para MAIÚSCULAS.
- **Declaração (`CadastroForm.Layout.cs`):**

```
C#
private readonly MaskedTextBox mskTelefone = new()
{
    Mask = "(00) 00000-0000", // Exemplos: "00000-000" (CEP), "000.000.000-00"
(CPF)
    Dock = DockStyle.Fill,
    TabIndex = 2,
    Margin = new Padding(3, 3, 24, 3)
};
```

- **Propriedades e Métodos Úteis:**
  - **`Text`**: Retorna o texto formatado (ex: `"(33) 98888-7777"`).
  - **`MaskCompleted`** (`bool`): Retorna `true` somente se o usuário preencheu **todos** os dígitos obrigatórios da máscara! (**Perfeito para usar no `ValidarCampos()`**).
  - **`Clear()`**: Limpa o campo (no `PrepararNovo()`).
- **Exemplo de Validação e Leitura (`CadastroForm.cs`):**

```C#
// Em ValidarCampos():
if (!mskTelefone.MaskCompleted)
{
    erros.SetError(mskTelefone, "Preencha o telefone completo.");
    mskTelefone.Focus();
    return false;
}

// Em Salvar():
string telefone = mskTelefone.Text;
```

1.2. `RichTextBox` (Texto longo ou formatado)

- **Finalidade (Slide 3):** Receber textos longos (várias linhas), como observações, descrições detalhadas ou conteúdo com formatação.
- **Declaração (`CadastroForm.Layout.cs`):**

```C#
private readonly RichTextBox rtbObservacoes = new()
{
    Dock = DockStyle.Fill,
    Height = 80,              // Altura caso a linha do TableLayoutPanel seja AutoSize
    MaxLength = 500,          // Limite de caracteres
    ReadOnly = false,         // true = apenas leitura
    ScrollBars = RichTextBoxScrollBars.Vertical, // Barra de rolagem vertical
    TabIndex = 5,
    Margin = new Padding(3, 3, 24, 3)
};
```

- **Leitura, Escrita e Limpeza:**
  - **Ler texto simples:** `string obs = rtbObservacoes.Text.Trim();`
  - **Carregar na edição:** `rtbObservacoes.Text = alunoEmEdicao.Observacoes;`
  - **Limpar no `PrepararNovo()`:** `rtbObservacoes.Clear();`
- **Evento principal:** `rtbObservacoes.TextChanged += ...;`

1.3. `LinkLabel` (Texto clicável estilo Hyperlink)

- **Finalidade (Slide 3):** Exibir um texto sublinhado clicável para abrir uma página, janela de ajuda ou recurso relacionado.

- **Declaração (`CadastroForm.Layout.cs`):**

```
C#
private readonly LinkLabel lnkAjuda = new()
{
    Text = "Consultar regulamento do curso",
    AutoSize = true,
    TabIndex = 6
};
```

- **Evento (`ConectarEventos`):** Usa o evento **`LinkClicked`** (em vez de `Click` comum):

```
C#
lnkAjuda.LinkClicked += (s, e) =>
{
    lnkAjuda.LinkVisited = true; // Muda a cor do link para roxo (visitado)
    MessageBox.Show("Regulamento: Presença mínima de 75%.", "Ajuda",
MessageBoxButtons.OK, MessageBoxIcon.Information);
};
```

---

2. Escolhas Múltiplas (`ListBox`, `CheckedListBox`)

2.1. `ListBox` (Lista de opções visíveis)

- **Finalidade (Slide 4):** Exibir várias opções visíveis ao mesmo tempo em uma caixa; aceita seleção única (`One`) ou múltipla (`MultiSimple` / `MultiExtended`).
- **Declaração e Preenchimento (`CadastroForm.Layout.cs`):**

```
C#
private readonly ListBox lstTurnos = new()
{
    Dock = DockStyle.Fill,
    Height = 70,
    SelectionMode = SelectionMode.One, // Seleção única (padrão)
    TabIndex = 3
};

// Dentro de MontarTela():
lstTurnos.Items.AddRange(new object[] { "Matutino", "Vespertino", "Noturno" });
```

- **Manipulação e Leitura (`CadastroForm.cs`):**
  - **Validar se selecionou (Seleção Única):** `if (lstTurnos.SelectedIndex < 0) { /* Erro */ }`

    - **Ler item escolhido:** `string turno = lstTurnos.SelectedItem?.ToString() ?? "";`
    - **Limpar seleção no `PrepararNovo()`:** `lstTurnos.ClearSelected();` (ou `lstTurnos.SelectedIndex = -1;`)
    - **Carregar na edição:** `lstTurnos.SelectedItem = alunoEmEdicao.Turno;`
- **Evento:** `lstTurnos.SelectedIndexChanged += ...;`

2.2. `CheckedListBox` (Lista com caixinhas de marcação independentes)

- **Finalidade (Slide 4):** Lista em que cada opção tem sua própria caixinha (`CheckBox`) independente. Ideal quando o aluno/produto pode ter **várias** características ao mesmo tempo (ex: Documentos entregues, Idiomas, Tags).
- **Declaração e Preenchimento (`CadastroForm.Layout.cs`):**

```
C#
private readonly CheckedListBox clbDocumentos = new()
{
    Dock = DockStyle.Fill,
    Height = 80,
    CheckOnClick = true, // Marca/desmarca com 1 clique só (sem precisar clicar 2 vezes!)
    TabIndex = 4
};

// Dentro de MontarTela():
clbDocumentos.Items.AddRange(new object[] { "CPF", "RG", "Histórico Escolar", "Foto 3x4" });
```

- **Como Ler, Limpar e Carregar (`CadastroForm.cs`):**
    - **Quantidade de itens marcados:** `int qtdMarcados = clbDocumentos.CheckedItems.Count;`
    - **Ler todos os itens marcados para uma `List<string>` ou `string`:**

```
C#
List<string> docsSelecionados = new();
foreach (var item in clbDocumentos.CheckedItems)
{
    docsSelecionados.Add(item.ToString() ?? "");
}
// Se quiser juntar tudo em uma string separada por vírgula:
string resumoDocs = string.Join(", ", docsSelecionados);
```

    - **Desmarcar tudo no `PrepararNovo()`:**

```C#
for (int i = 0; i < clbDocumentos.Items.Count; i++)
{
    clbDocumentos.SetItemChecked(i, false);
}
clbDocumentos.ClearSelected();
```

---

3. Escalas e Progresso (`TrackBar`, `ProgressBar`)

3.1. `TrackBar` (Controle deslizante / Slider)

- **Finalidade (Slide 5):** Escolher um valor numérico contínuo aproximado arrastando um pino (ex: nível de satisfação, volume, urgência de 0 a 10).
- **Declaração (`CadastroForm.Layout.cs`):**

```C#
private readonly TrackBar trkNivel = new()
{
    Dock = DockStyle.Fill,
    Minimum = 0,          // Valor mínimo
    Maximum = 10,         // Valor máximo
    Value = 5,            // Posição inicial
    TickFrequency = 1,    // Marquinhas na régua a cada 1 unidade
    AutoSize = true,
    TabIndex = 5
};
```

- **Leitura e Escrita (`Value` é `int` direta, não precisa de cast!):**
  - **Ler:** `int nivel = trkNivel.Value;`
  - **Alterar:** `trkNivel.Value = 0;`
- **Evento:** `trkNivel.ValueChanged += (s, e) => lblValorNivel.Text = $"Nível: {trkNivel.Value}";` (ou `trkNivel.Scroll`).

3.2. `ProgressBar` (Barra de Progresso)

- **Finalidade (Slide 5):** Comunicar visualmente o andamento de uma operação ou meta (ex: lotação de uma turma, preenchimento de ficha).
- **Declaração (`CadastroForm.Layout.cs`):**

```C#
private readonly ProgressBar prgLotacao = new()
{
    Dock = DockStyle.Fill,
    Minimum = 0,
    Maximum = 100,
    Value = 0,
    Style = ProgressBarStyle.Blocks // Ou ProgressBarStyle.Continuous
};
```

- **Como atualizar o valor no código:**
  - `prgLotacao.Value = 50;` (Atenção: nunca atribua um valor menor que `Minimum` ou maior que `Maximum`, senão lança exceção!).
  - `prgLotacao.Increment(10);` (Avança a barra em `10` pontos com segurança).

---

4. Retorno ao Usuário: `ToolTip` (Dica Contextual)

- **Finalidade (Slide 6):** Componente **não visual** (igual ao `ErrorProvider`) que mostra um balãozinho de dica quando o usuário passa o mouse por cima de qualquer controle.
- **Como usar (`CadastroForm.Layout.cs`):**

```C#
// 1. Declarar como campo da classe:
private readonly ToolTip dicas = new();

// 2. Dentro de MontarTela(), associar textos aos controles:
dicas.SetToolTip(txtNome, "Digite o nome completo sem abreviações.");
dicas.SetToolTip(btnSalvar, "Salva os dados na memória (Enter).");

// 3. No Dispose(bool disposing) em CadastroForm.cs:
if (disposing)
{
    erros.Dispose();
    dicas.Dispose();
}
```

---

5. Contêineres Extras (`GroupBox`, `Panel`, `TabControl`, `SplitContainer`)

5.1. `GroupBox` (Agrupador com Borda e Título)

- **Finalidade (Slides 8, 14, 16 e 17):** Muito cobrado nas imagens da Aula 6! Cria uma moldura com um título escrito na borda superior (ex: `"Modalidade"`, `"Dados da inscrição"`). É o melhor lugar para colocar `RadioButtons` ou agrupar todo o formulário.
- **Como usar com `RadioButtons` (como na imagem da Ficha de Inscrição do Slide 17):**

```
C#
private readonly GroupBox grpModalidade = new()
{
    Text = "Modalidade",
    AutoSize = true,
    Dock = DockStyle.Fill,
    Padding = new Padding(6),
    TabIndex = 4
};

// Dentro de MontarTela():
// Coloca o FlowLayoutPanel (com rdbPresencial e rdbRemota) dentro do GroupBox!
painelModalidade.Controls.AddRange(new Control[] { rdbPresencial, rdbRemota });
grpModalidade.Controls.Add(painelModalidade);

// Adiciona o GroupBox na célula do TableLayoutPanel:
layout.Controls.Add(grpModalidade, 1, 4);
```

5.2. `Panel` (Painel Simples com Rolagem)

- **Finalidade (Slide 8):** Agrupa controles sem mostrar borda/título por padrão e permite barra de rolagem automática quando a tela fica pequena.
- **Declaração:**

```
C#
private readonly Panel pnlConteudo = new()
{
    Dock = DockStyle.Fill,
    AutoScroll = true,                  // Habilita barra de rolagem se faltar
espaço!
```

```
    BorderStyle = BorderStyle.None      // Ou BorderStyle.FixedSingle
};
// Para colocar controles dentro: pnlConteudo.Controls.Add(meuLayout);
```

5.3. `TabControl` e `TabPage` (Organização em Abas)

- **Finalidade (Slide 8):** Separar o conteúdo da tela em abas/páginas selecionáveis (ex: Aba 1 = `"Cadastro"`, Aba 2 = `"Consulta / Lista"`).
- **Como montar em `MontarTela()`:**

```
C#
var abas = new TabControl { Dock = DockStyle.Fill };

var abaCadastro = new TabPage { Text = "Dados Cadastrais", Padding = new
Padding(10) };
var abaLista = new TabPage { Text = "Registros Salvos", Padding = new Padding(10)
};

abaCadastro.Controls.Add(layoutFormulario); // Coloca o TableLayoutPanel na Aba 1
abaLista.Controls.Add(grade);               // Coloca o DataGridView na Aba 2

abas.TabPages.AddRange(new TabPage[] { abaCadastro, abaLista });
Controls.Add(abas);
```

5.4. `SplitContainer` (Divisor de Janela Redimensionável)

- **Finalidade (Slide 8):** Divide a janela em duas áreas (`Panel1` e `Panel2`) separadas por uma barra móvel que o usuário pode arrastar.
- **Como montar em `MontarTela()`:**

```
C#
var divisor = new SplitContainer
{
    Dock = DockStyle.Fill,
    Orientation = Orientation.Horizontal, // Horizontal = Cima/Baixo; Vertical =
Esquerda/Direita
    SplitterDistance = 220                // Tamanho inicial do Panel1 em pixels
};

divisor.Panel1.Controls.Add(layoutFormulario); // Parte de cima (ou esquerda)
divisor.Panel2.Controls.Add(grade);            // Parte de baixo (ou direita)
```

```
Controls.Add(divisor);
```

---

6. Listas e Árvores (`ListView`, `TreeView`)

6.1. `ListView` (Lista em Colunas/Detalhes ou Ícones)

- **Finalidade (Slide 7):** Apresentar itens em modo de lista, ícones ou detalhes com colunas (`View.Details`). Diferente do `DataGridView`, não usa `DataSource` direto: os itens são adicionados como `ListViewItem`.
- **Como configurar e usar:**

```
C#
private readonly ListView lvwAlunos = new()
{
    Dock = DockStyle.Fill,
    View = View.Details,        // Exibe formato de tabela com colunas
    FullRowSelect = true,       // Seleciona a linha toda
    GridLines = true            // Mostra linhas de grade
};

// Em MontarTela() – criar colunas (Nome, Largura em px):
lvwAlunos.Columns.Add("Nome", 200);
lvwAlunos.Columns.Add("Curso", 150);

// Como adicionar uma linha manualmente via código:
var item = new ListViewItem(nome); // Coluna 0
item.SubItems.Add(curso);          // Coluna 1
lvwAlunos.Items.Add(item);
```

6.2. `TreeView` e `TreeNode` (Estrutura Hierárquica / Árvore)

- **Finalidade (Slide 7):** Representar dados hierárquicos com nós pais e filhos, como pastas, departamentos ou categorias e subcategorias.
- **Como configurar e usar:**

```
C#
private readonly TreeView tvwCategorias = new()
{
    Dock = DockStyle.Fill
```

```
};

// Em MontarTela() – adicionando Nós Pais e Filhos:
var noComputacao = new TreeNode("Computação");
noComputacao.Nodes.Add("Sistemas de Informação");
noComputacao.Nodes.Add("Ciência da Computação");

var noAgrarias = new TreeNode("Ciências Agrárias");
noAgrarias.Nodes.Add("Agronomia");
noAgrarias.Nodes.Add("Engenharia Florestal");

tvwCategorias.Nodes.AddRange(new TreeNode[] { noComputacao, noAgrarias });
tvwCategorias.ExpandAll(); // Abre todos os ramos da árvore inicialmente

// Evento ao selecionar um nó (AfterSelect):
tvwCategorias.AfterSelect += (s, e) =>
{
    string selecionado = e.Node?.Text ?? "";
};
```

---

7. Diálogos Prontos Extras (`SaveFileDialog`, `FolderBrowserDialog`, `ColorDialog`, `FontDialog`)

Todos seguem o mesmo padrão do `OpenFileDialog` do Slide 10: instancia com `using var`, verifica se `ShowDialog() == DialogResult.OK` e lê a propriedade resultante:

```
C#
// 1. SaveFileDialog (Escolher onde salvar um arquivo)
using var sfd = new SaveFileDialog
{
    Filter = "Arquivo de Texto|*.txt|Todos os arquivos|*.*",
    Title = "Salvar Relatório"
};
if (sfd.ShowDialog() == DialogResult.OK)
{
    string caminhoSalvar = sfd.FileName;
    File.WriteAllText(caminhoSalvar, "Conteúdo do arquivo");
}

// 2. FolderBrowserDialog (Selecionar uma pasta/diretório)
using var fbd = new FolderBrowserDialog
{
    Description = "Selecione a pasta de backup"
```

```
};
if (fbd.ShowDialog() == DialogResult.OK)
{
    string pastaEscolhida = fbd.SelectedPath;
}

// 3. ColorDialog (Escolher uma cor na paleta do Windows)
using var cd = new ColorDialog();
if (cd.ShowDialog() == DialogResult.OK)
{
    lblTituloForm.ForeColor = cd.Color; // Aplica a cor escolhida!
}

// 4. FontDialog (Escolher fonte, tamanho e estilo)
using var fd = new FontDialog();
if (fd.ShowDialog() == DialogResult.OK)
{
    txtNome.Font = fd.Font; // Aplica a fonte escolhida!
}
```

---

8. Tempo, Imagens e Segundo Plano (`Timer`, `ImageList`, `NotifyIcon`, `BackgroundWorker`)

8.1. `Timer` (Relógio / Tarefa Periódica na Interface)

- **Finalidade (Slide 11):** Disparar o evento `Tick` repetidamente em intervalos de tempo definidos (em milissegundos). Exemplo clássico: atualizar um relógio na `StatusStrip` a cada 1 segundo (`1000` ms).
- **Atenção do Slide 11:** Usar `System.Windows.Forms.Timer` e **nunca** executar operações pesadas/demoradas dentro do `Tick` para não travar a janela.
- **Como configurar e usar:**

```
C#
// Campo da classe:
private readonly System.Windows.Forms.Timer relogio = new()
{
    Interval = 1000, // 1000 ms = 1 segundo
    Enabled = true   // Já começa rodando (ou chame relogio.Start() no construtor)
};

// Em ConectarEventos():
relogio.Tick += (s, e) =>
```

```
{
    lblStatusHora.Text = DateTime.Now.ToString("HH:mm:ss");
};

// No Dispose(bool disposing):
if (disposing)
{
    relogio.Dispose();
}
```

8.2. `ImageList`, `NotifyIcon` e `BackgroundWorker` (Resumo Prático e Conceitual - Slide 11)

- **`ImageList`**:
  - Coleção não visual que guarda ícones/imagens pequenas associadas a índices (`0`, `1`...) ou chaves (`"salvar"`, `"excluir"`) para serem usadas em botões de `ToolStrip`, `ListView`, `TreeView` ou `TabControl`.
  - Propriedade principal: `ImageSize = new Size(16, 16);` e método `Images.Add("chave", imagem);`.
- **`NotifyIcon`**:
  - Exibe um ícone do programa na área de notificação do Windows (perto do relógio da barra de tarefas, o *System Tray*).
  - Propriedades principais: `Icon = SystemIcons.Application;`, `Text = "Sistema em execução";`, `Visible = true;`.
  - Método para exibir notificação do Windows: `iconeBandeja.ShowBalloonTip(3000, "Título", "Mensagem", ToolTipIcon.Info);` (também deve ser descartado no `Dispose`).
- **`BackgroundWorker`**:
  - Componente para executar tarefas demoradas em segundo plano sem congelar a interface gráfica.
  - Eventos principais:
    1. `DoWork`: Onde roda o código pesado em outra thread (não pode mexer nos controles da tela aqui dentro!).
    2. `ProgressChanged`: Atualiza uma `ProgressBar` na tela (exige `WorkerReportsProgress = true`).
    3. `RunWorkerCompleted`: Dispara quando o trabalho termina; aqui você pode atualizar os controles da tela com segurança.
  - Para iniciar: `trabalhador.RunWorkerAsync();`.